

COMP90041

Programming and Software Development

Lecture 10 - Generics and Collections Framework

Danny Xu

The University of Melbourne acknowledges the Traditional Owners of the unceded land on which we work, learn and live: the Wurundjeri Woi-wurrung and Bunurong peoples (Burnley, Fishermans Bend, Parkville, Southbank and Werribee campuses), the Yorta Yorta Nation (Dookie and Shepparton campuses), and the Dja Dja Wurrung people (Creswick campus).

The University also acknowledges and is grateful to the Traditional Owners, Elders and Knowledge Holders of all Indigenous nations and clans who have been instrumental in our reconciliation journey.

We recognise the unique place held by Aboriginal and Torres Strait Islander peoples as the original owners and custodians of the lands and waterways across the Australian continent, with histories of continuous connection dating back more than 60,000 years. We also acknowledge their enduring cultural practices of caring for Country.

We pay respect to Elders past, present and future, and acknowledge the importance of Indigenous knowledge in the Academy. As a community of researchers, teachers, professional staff and students we are privileged to work and learn every day with Indigenous colleagues and partners.

WARNING

This material has been reproduced and communicated to you by or on behalf of the University of Melbourne in accordance with section 113P of the *Copyright Act 1968 (Act)*.

The material in this communication may be subject to copyright under the Act.

Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice

Overview



In this part of the lecture, you will learn about:

- **Generics**
- **Collections Framework**
- **ArrayLists**



THE UNIVERSITY OF
MELBOURNE

Generics

What are Generics?



- Generics allow you to write flexible, reusable code that can work with any data type.
- They enable **type-safe** operations while maintaining **code generality**.
- They are somewhat different to what Interfaces and Inheritance provide.

Feature	Generics	Inheritance	Interfaces
Purpose	Enable code reusability with type safety	Enable reuse of behavior and properties	Define a contract that classes must implement
Type Focus	Works with any data type (type parameters)	Works with specific class hierarchies	Works with any class implementing the interface

Generics Example



- Suppose you want to sort a list of movies, books, students. Non-related means No inheritance. We can implement a sorting interface but have to write code 3 times in 3 classes.
- Generics – Can write one class with one method using generics and reduce duplication of code.
- How do we sort? We need to compare what comes before.
- How do we compare? Implement Interface called Comparable.

Generics Example



```
class Student implements Comparable<Student> {
    int studentId;
    String name;
    @Override
    public int compareTo(Student other) {
        return Integer.compare(this.studentId, other.studentId ); // sort by grade
    }
}

class Book implements Comparable<Book> {
    int year;
    String name;
    @Override
    public int compareTo(Book other) {
        return Integer.compare(this.year, other.year); // sort by year
    }
}

class Movie implements Comparable<Movie> {
    double rating;
    String name;
    @Override
    public int compareTo(Movie other) {
        return Double.compare(this.rating, other.rating); // sort by rating
    }
}
```


Generic Example

T here means any type
– Book, Movie, Student



```
class ArraySorter<T extends Comparable<T>> {  
    public void sortArray(T[] array) {  
        int n = array.length;  
        for (int i = 0; i < n - 1; i++) {  
            for (int j = 0; j < n - 1 - i; j++) {  
                if (array[j].compareTo(array[j + 1]) > 0) {  
                    // swap elements  
                    T temp = array[j];  
                    array[j] = array[j + 1];  
                    array[j + 1] = temp;  
                }  
            }  
        }  
    }  
}
```

T has to implement
Comparable

The same type will
be used everywhere

Generics Types



- T is a **type parameter**; <T> indicates a **generic method or class**.
- Generics allow defining **classes and methods** that operate on **any reference type**.
- When a specific type is provided, it produces a **concrete version** of the class or method.
- Generics improve **code reusability** and **type safety** at compile time.
- A **generic (parameterized) class** includes the type parameter in angle brackets after the class name.
- The type parameter can be used in:
 - **Instance variables**
 - **Method parameters**
 - **Return types**

Generics Naming Conventions



- T: Type (general purpose)
- E: Element (used in collections like `ArrayList<E>`)
- K: Key (used in maps like `HashMap<K, V>`)
- V: Value
- N: Number
- S, U, V: Second, third, fourth types, etc.

How do you use these classes?



```
Book[] arrBooks = new Book[]{new Book("ABC",2021), new Book("DEF",2023)};  
ArraySorter<Book> bookSorter = new ArraySorter<>(); // create a new object  
bookSorter.sortArray(arrBooks); // pass an array of Books
```

```
Movie[] arrMovie = new Movie[]{new Movie("ABC",2.5), new Movie("DEF",3.5)};  
ArraySorter<Movie> movieSorter = new ArraySorter<>(); // create a new object  
movieSorter.sortArray(arrMovie); // pass an array of movies
```

- Every time you create an object of ArraySorter, you pass what type it would take. At a time only one type is passed here as per the definition.

Multiple Types : Generics



- You can also pass multiple types in a generic.
- You need to use different symbols to represent different types a class will take.

```
public class TwoTypePair<T1, T2> {  
    private T1 first;  
    private T2 second;  
  
    public T2 findValueByFirst(TwoTypePair<T1, T2>[] pairList, T1 value){  
        for(TwoTypePair<T1,T2> p: pairList){  
            if(p.first.equals(value)){  
                return p.second;  
            }  
        }  
    }  
}
```

Bounds for Type



- Can a generic class work for everything? Answer is No.
- Certain functionality may not be applicable to every class present.
- Thus we can define a bound on the type T used.
- Remember ArraySorter only takes types T that has interface comparable implemented. Without compareTo method implemented, the ArraySorter can not sort.
- Type can extend
 - A class `public class ExClass<T extends Class1>`
 - An interface `public class ExClass<T extends Interface1>`
 - A class and interface `public class ExClass<T extends Class1 & Interface1>`
 - A class and multiple interface `public class ExClass<T extends Class1 & Interface1 & Interface2>`

Generic Bounds on Methods



- If you create a class `Generic`, static methods defined cannot directly use the class type parameter `T` because static members belong to the class, not instances.
- Sometimes you want to add static methods for utility functions.
- In this case, one must explicitly attach the type to methods instead of classes.
- Different methods can have different types attached. See below

This T is
type
defined

```
public class Utility {  
    public static <T> T getMidPoint(T[] a) {  
        return a[a.length/2];  
    }  
    public static <T1, T2> boolean isSameClass(T1 a, T2 b)  
    {  
        return (a.getClass() == b.getClass());  
    }  
}
```

This T is
return
type

Advanced Non-Examinable Topics :

Generic Inheritance + Interfaces



- You can have a hierarchy of classes that can have generics type defined. Eg -
 - `Pair<T1, T2>`
 - `UnorderedPair<T1,T2>` extends `Pair<T1,T2>`
 - `OrderedPair<T1,T2>` extends `Pair<T1,T2>`
- You can also have Generic Interfaces and have generic classes implement them like

```
public interface Pair<K, V> {  
    K getKey();  
    V getValue();  
}
```


Pitfalls: Advanced Non-Examinable



- Cannot use type parameters with `new` in a Generic class:
 - `new T()` and `new T[10]` are illegal.
 - Workarounds exist but are complex (e.g., using reflection or object arrays).
- Cannot create arrays of generic types:
 - `new Pair<String>[10]` is not allowed.
 - Use `Object[]` and cast when retrieving.
- Generic classes cannot extend `Throwable`:
 - Cannot create a generic exception class like `class GEx<T> extends Exception`.



THE UNIVERSITY OF
MELBOURNE

Collections Framework

Collections



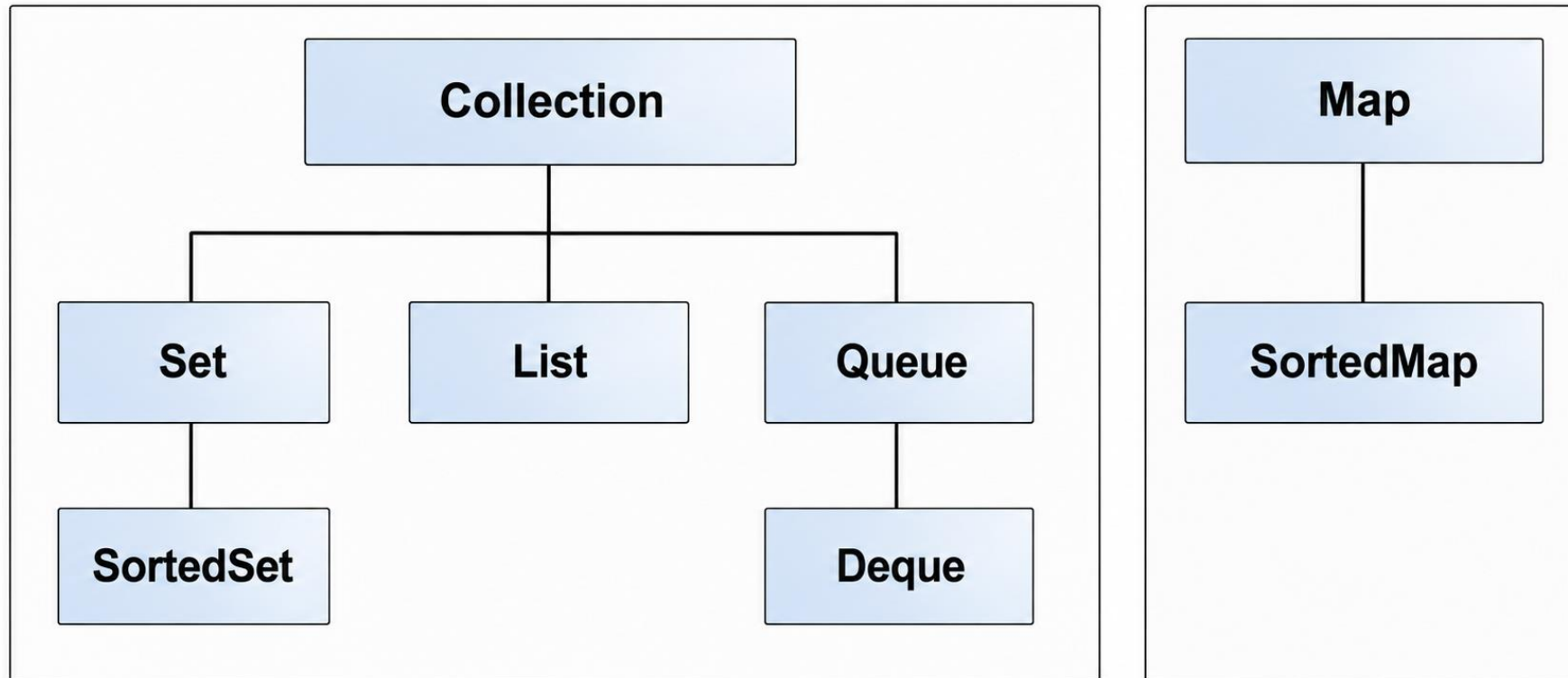
- What is a Collection?
 - An object that groups multiple elements into a single unit. Not Arrays.
 - Used for storing, retrieving, and manipulating grouped data (e.g., list of cards, phonebook).
- What is a Collections Framework?
 - Unified architecture for handling collections.
 - Composed of:
 - Interfaces – abstract types for various collections (List, Set, Map, etc.).
 - Implementations – concrete classes (ArrayList, HashSet, HashMap, etc.).
 - Algorithms – reusable methods like sorting and searching (e.g., `Collections.sort()`).

Benefits of Collections



- **Reduces programming effort** – provides ready-to-use data structures and algorithms.
- **Improves speed and quality** – high-performance, interchangeable implementations.
- **Enables API interoperability** – collections passed between unrelated systems easily.
- **Simplifies learning and usage** – consistency across APIs reduces errors and effort.
- **Simplifies API design** – reuse standard interfaces instead of inventing new ones.
- **Fosters software reuse** – algorithms and data structures are plug-and-play.

Collections Interfaces

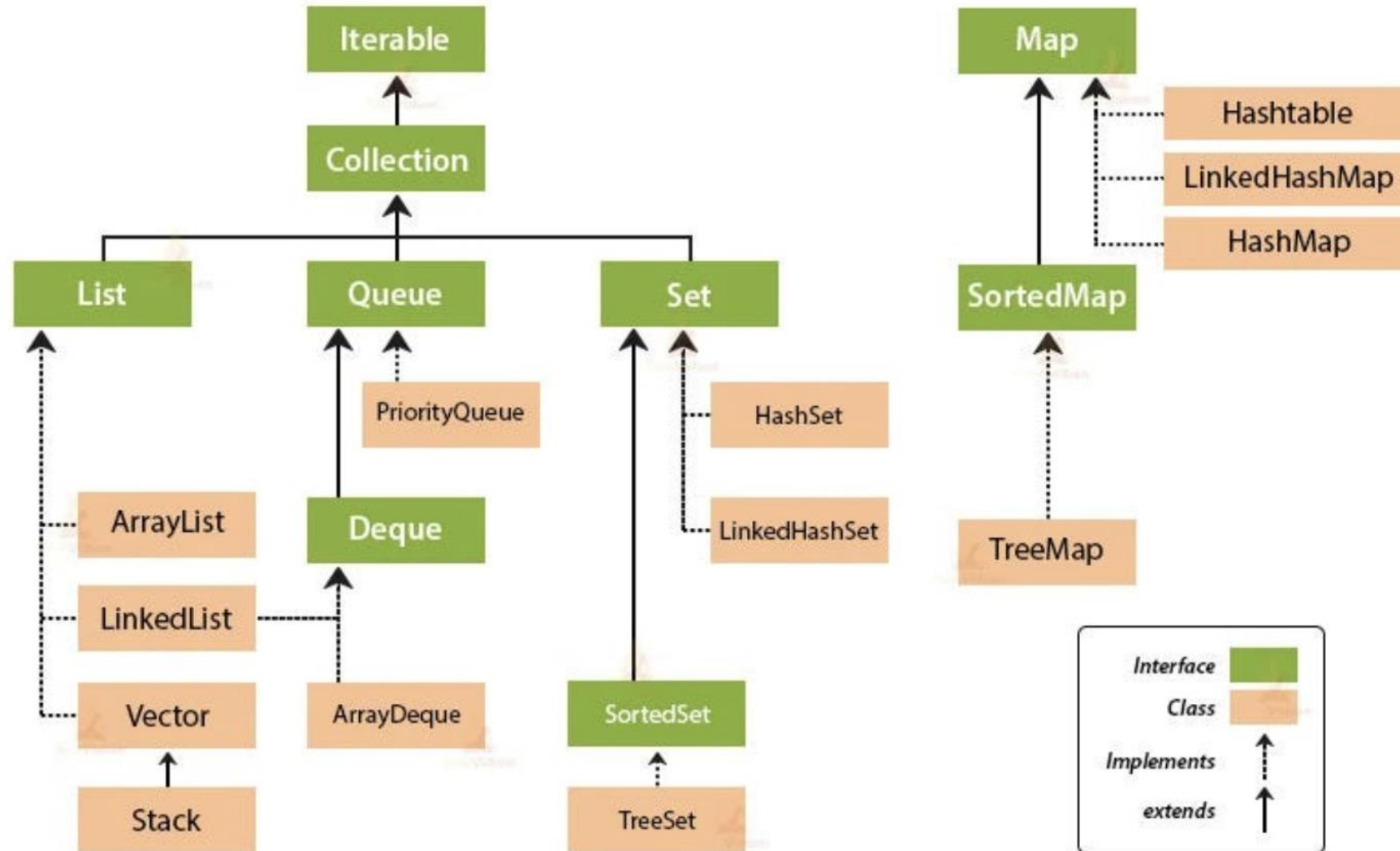


Interfaces

- Collection Interface
 - Represents a group of elements
 - May allow duplicates or be ordered/unordered.
- Set Interface (Set<E>)
 - Cannot contain duplicate elements.
 - Models mathematical sets
- List Interface (List<E>)
 - Ordered collection (sequence).
 - Allows duplicate elements.
 - Access via index; insertion order preserved.
- Map Interface (Map<K, V>)
 - Maps unique keys to values/elements.
 - Each key maps to only one value. Value could be arrays.



Collection Framework Hierarchy in Java



Collection Classes



Feature	List	Map	Set
Order	Elements maintain the order of insertion.	Elements do not have a defined order.	Elements do not have a defined order.
Duplicates	Allows duplicate elements.	Does not allow duplicate keys.	Does not allow duplicate elements.
Key-Value Pairs	No key-value pairs.	Uses key-value pairs to store data.	No key-value pairs.
Null Values	Allows null elements (both value and index).	Allows a single null key and any number of null values.	Allows at most one null element.
Common Implementations	ArrayList, LinkedList, Vector	HashMap, TreeMap, LinkedHashMap	HashSet, LinkedHashSet, TreeSet
Use Cases	Ordered collections (sequences).	Storing key-value relationships.	Unique elements (removing duplicates).
Access by Index	Yes, elements can be accessed by index.	No, elements cannot be accessed by index.	No, elements cannot be accessed by index.
Iteration	Can iterate through elements in order.	Can iterate through key-value pairs.	Can iterate through elements, but order is not guaranteed.
Searching	Can search for elements by index or value.	Can search for values using keys.	Can check if a specific element exists.

Iterator Interface



- As collections are a group of data, we need to iterate over them. The Collection framework provides many ways to do it.

```
public interface Iterator<E> {  
    boolean hasNext();  
    E next();  
    void remove();  
}
```

- Collection classes already implement this. You just have to use it.

```
ArrayList<String> collection = new ArrayList();  
collection.add("Apple");  
collection.add("Banana");  
collection.add("Orange");  
  
Iterator<String> iterator = collection.iterator();  
while (iterator.hasNext()) { // looking for the next element if existed  
    String element = iterator.next(); // finding the next  
    element          System.out.println(element);  
}
```

Comparator Interface



- Comparator Interfaces provides a comparison function, which imposes a total ordering on some collection of objects.

```
public static void main(String[] args) {
    List<Person> people = new ArrayList<>();
    people.add(new Person("Alice", 30));
    people.add(new Person("Bob", 25));
    people.add(new Person("Charlie", 35));

    // Sort the list using Collections.sort and the custom comparator
    Collections.sort(people, new AgeComparator());

    System.out.println("People sorted by age (ascending):");
    for (Person person : people) {
        System.out.println(person);
    }
}

class AgeComparator implements Comparator<Person> {
    @Override
    public int compare(Person p1, Person p2) {
        return Integer.compare(p1.getAge(), p2.getAge());
    }
}
```

Comparator vs Comparable



- Comparator implemented by a comparator class that defines the compare method
 - This comparator class can be shared by many many different types.
 - Usually can be implemented for a type that defines similar features.
 - Example – an AgeComparator can be used for Person, Wine, Student, Tree, Dog. CompareTo method is only defined once for the type.
- Comparable is implemented by individual classes.
 - Different classes will have different comparisons.
 - Makes sense to implement custom compareTo methods.
 - Example – a tree can be compared based on height and age both.



THE UNIVERSITY OF
MELBOURNE

ArrayLists

ArrayList



- ArrayLists are much more used than arrays.
- ArrayLists can dynamically shrink or grow. **No resizing required.**
- ArrayLists have same properties like an array
 - Can hold a collection of same type of objects
 - Insertion order is maintained
- ArrayList differs from arrays -
 - They are less efficient because every insertions internally resizes at some point which is not in your control
 - ArrayList uses the generic notation `<>` and not the square brackets `[ ]`
 - ArrayList can only create collection of objects and not primitive types. Use Wrapper classes instead – `ArrayList<Integer>` instead of `int[ ]`

ArrayList Operations



- **Create a new ArrayList**
 - `ArrayList<String> list = new ArrayList<>();`
- **Add elements**
 - At the end: `list.add("Apple");`
 - At a specific index: `list.add(1, "Banana");`
- **Access an element**
 - `String item = list.get(0);` // gets the first item
 - `String item = list.get(5);` // gets the 6th item
- **Update (set) an element**
 - `list.set(2, "Orange");` // replaces third item
- **Get size**
 - `int size = list.size();`

ArrayList Operations



- **Remove an element**
 - By index: `list.remove(0);`
 - By object: `list.remove("Banana");`
- **Check if empty**
 - `list.isEmpty();`
- **Check if contains an element**
 - `list.contains("Apple");`
- **Loop through elements**
 - ```
for (String s : list) {
 System.out.println(s);
}
```



THE UNIVERSITY OF  
MELBOURNE

# Live Coding



# Live Coding



- Printable Class using Generics
  - Prints to the standard output
  - Prints to a file
- To-Do List using ArrayLists
  - Add tasks to todo list
  - Print the tasks
  - Delete when done
  - Insert in middle of the list

# Additional Reading Resources



- WALTER, S. Absolute Java, Global Edition. [Harlow]: Pearson, 2016. (Chapter 14)
- SCHILDT, H. Java: The Complete Reference, 12th Edition: McGraw-Hill, 2022 (Chapter 14)
- WALTER, S. Absolute Java, Global Edition. [Harlow]: Pearson, 2016. (Chapter 16 )
- SCHILDT, H. Java: The Complete Reference, 12th Edition: McGraw-Hill, 2022 (Chapter 20)

# Copyright Notice



(c) The University of Melbourne 2023

This material is protected by copyright. Unless indicated otherwise, the University of Melbourne owns the copyright subsisting in the work. Other than for the purposes permitted under the Copyright Act 1968, you are prohibited from downloading, republishing, retransmitting, reproducing or otherwise using any of the materials included in the work as standalone files. Sharing University teaching materials with third-parties, including uploading lecture notes, slides or recordings to websites constitutes academic misconduct and may be subject to penalties. For more information: [Academic Integrity at the University of Melbourne](#)

Requests and enquiries concerning reproduction and rights should be addressed to the University Copyright Office, The University of Melbourne: [copyright-office@unimelb.edu.au](mailto:copyright-office@unimelb.edu.au)

The University of Melbourne (Australian University)  
PRV12150/CRICOS 00116K

# See you next time!



THE UNIVERSITY OF  
MELBOURNE